

19

The future of AI

This chapter covers

- The limitations of deep learning
- The nature of intelligence
- What's missing from current approaches
- What the future might look like

To use a tool appropriately, you should not only understand what it *can* do but also be aware of what it *can't* do. I'm going to present an overview of some key limitations of deep learning. Then, I'll offer some speculative thoughts about the future evolution of AI and what it would take to get to human-level general intelligence. This should be especially interesting to you if you'd like to get into fundamental research.

19.1 *The limitations of deep learning*

There are infinitely many things you can do with deep learning. But deep learning can't do *everything*. To use a tool well, you should be aware of its limitations, not just its strengths. So where does deep learning fall short?

19.1.1 Deep learning models struggle to adapt to novelty

Deep learning models are big, parametric curves fitted to large datasets. That's the source of their power—they're easy to train, and they scale really well, both in terms of model size and dataset size. But that's also a source of significant weaknesses. Curve fitting has inherent limitations.

First and foremost, a parametric curve is only capable of information storage—it's a kind of *database*. Recall our discussion of Transformers as an “interpolative database” from chapter 15? Second, crucially, this database is *static*. The model's parameters are determined during a distinct “training time” phase. Afterward, these parameters are frozen, and this fixed version is used during “inference time” for making predictions on new data.

The only thing you can do with a static database is information retrieval. And that's exactly what deep learning models excel at: recognizing or generating patterns highly similar to those encountered during training. The flip side is that they are inherently poor at *adaptation*. The database is backward-looking—it fits past data but can't handle a changing future. At inference time, you'd better hope that the situations the model faces are part of the training data distribution, because otherwise, the model will break down. A model trained on ImageNet will classify a leopard-print sofa as an actual leopard, for instance—sofas were not part of its training data.

This also applies to the largest of generative models. In recent years, the rise of large language models (LLMs) and their application to programming assistance and reasoning-like problems has provided extensive empirical proof of this. Despite frequent claims that LLMs can perform *in-context learning* to pick up new skills from just a few examples, there is overwhelming evidence that what they're actually doing is fetching vector functions they've memorized during training and reapplying them to the task at hand. By learning to do next-token prediction across a web-sized text dataset, an LLM has collected millions of potentially useful mini text-processing programs, and it can easily be prompted into reusing them on a new problem. But show it something that has no direct equivalent in its training data, and it's helpless.

Take a look at the puzzle in figure 19.1. Did you figure out the solution? Good. It's not very hard, is it? But today, no state-of-the-art LLM or vision-language model can do this because this particular problem doesn't directly map to anything they've seen at training time—even after having been trained on the entire internet and then some. An LLM's ability to solve a given problem has nothing to do with problem complexity, and everything to do with *familiarity*—they will break their teeth on any sufficiently novel problem, no matter how simple.

This failure mode even applies to tiny variations of a pattern that an LLM encountered many times in its training data. For instance, for a few months after the release of ChatGPT, if you asked it, “What's heavier, 10 kilos of steel or one kilo of feathers?,” it would answer that they weigh the same. That's because the question “What's heavier, one kilo of steel or one kilo of feathers?” is found many times on the internet—as a trick question. The right answer, of course, is that they both weigh the same, so the GPT

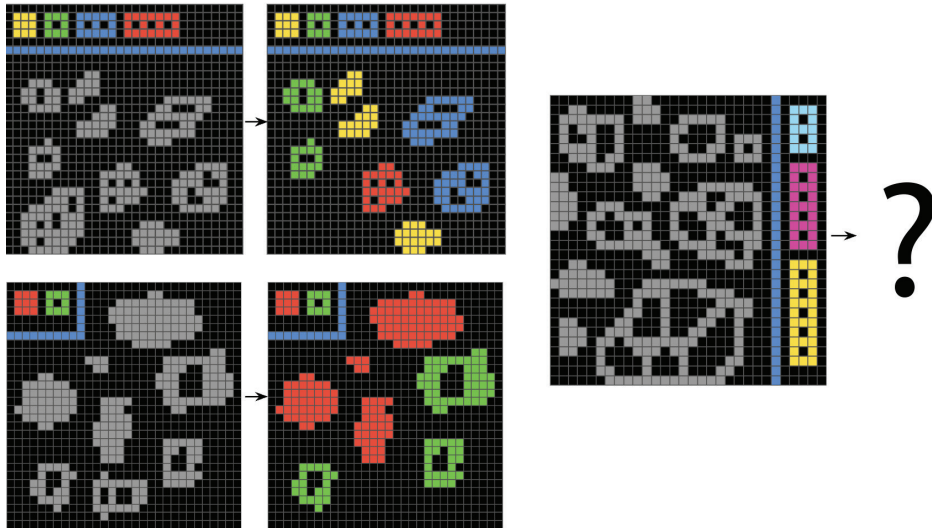


Figure 19.1 An easy yet novel puzzle

model would just repeat the answer it had memorized without paying any attention to the actual numbers in the query, or what the query really *meant*. Similarly, LLMs struggle to adapt to variations of the Monty Hall problem (see figure 19.2) and will tend to always output the canonical answer to the puzzle, which they've seen many times during training, regardless of whether it makes sense in context.



Suppose you're on a game show, and you're given the choice of three doors: Behind one door is a car; behind the others, goats. You pick a door, say No. 1, and the host, who knows what's behind the doors, opens door No. 1, and you see a car. It's a black BMW. He then says to you, "Do you want to change your choice?" Is it to your advantage to change your choice?



Yes, it is to your advantage to change your choice. This scenario is a classic probability puzzle known as the Monty Hall problem, and it has a counterintuitive solution.

Initially, when you picked door No. 1, you had a 1/3 chance of choosing the car and a 2/3

Figure 19.2 A variation of the Monty Hall problem

To note, these specific prompts were patched later on by special-casing them. Today, there are over 25,000 people who are employed full time to provide training data for LLMs by reviewing failure cases and suggesting better answers. LLM maintenance is a constant game of whack-a-mole where failing prompts are patched one at a time,

without addressing the more general underlying issue. Even already patched prompts will still fail if you make small changes to them!

19.1.2 Deep learning models are highly sensitive to phrasing and other distractors

A closely related problem is the extreme sensitivity of deep learning models to how their input is presented. For instance, image models are affected by *adversarial examples*, which are samples fed to a deep learning network that are designed to trick the model into misclassifying them. You're already aware that it's possible to do gradient ascent in input space to generate inputs that maximize the activation of some ConvNet filter—this is the basis of the filter visualization technique introduced in chapter 10.

Similarly, through gradient ascent, you can slightly modify an image to maximize the class prediction for a given class. By taking a picture of a panda and adding to it a gibbon gradient, we can get a neural network to classify the panda as a gibbon (see figure 19.3). This evidences both the brittleness of these models and the deep difference between their input-to-output mapping and our human perception.

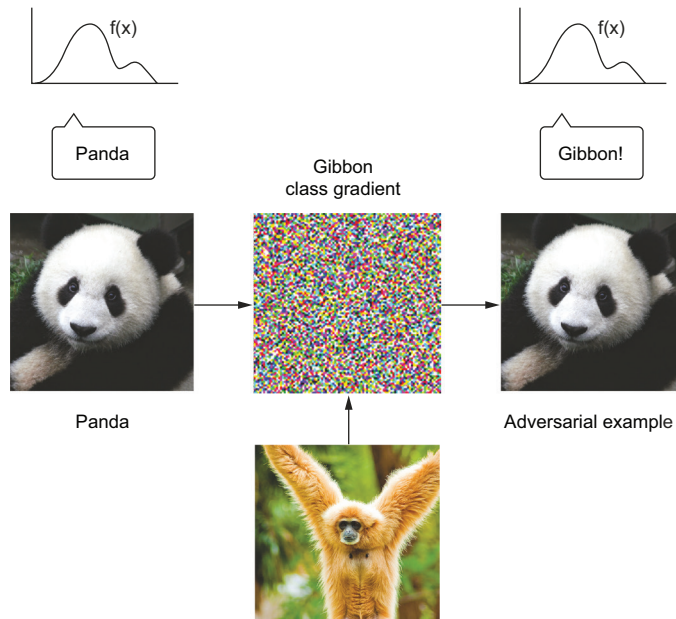


Figure 19.3 An adversarial example: imperceptible changes in an image can upend a model's classification of the image.

Similarly, LLMs suffer from an extremely high sensitivity to minor details in their prompts. Innocuous prompt modifications, such as changing place and people's

names in a text paragraph or variable names in a block of code, can significantly degrade LLM performance. Consider the well-known *Alice in Wonderland* riddle:¹

“Alice has N brothers and she also has M sisters. How many sisters does Alice’s brother have?”

The answer, of course, is $M + 1$ (Alice’s sisters plus Alice herself). For an LLM, asking the question with values commonly found in online instances of the riddle (like $N = 3$ and $M = 2$) will generally result in the correct answer, but try tweaking the values of M and N , and you will quickly get incorrect answers.

This oversensitivity to phrasing has given rise to the concept of *prompt engineering*. Prompt engineering is the art of formulating LLM prompts in a way that maximizes performance on a task. For instance, it turns out that adding the instruction “Please think step by step” to a prompt that involves reasoning can significantly boost performance. The term *prompt engineering* is a very optimistic framing of the underlying issue: “Your models are better than you know! You just need to use them right!” A more negative framing would be to point out that for any query that seems to work, there’s a range of minor changes that have the potential to tank performance. To what extent do LLMs understand something if you can break their understanding with simple rewordings?

What’s behind this phenomenon is that an LLM is a big parametric curve—a medium for storing knowledge and programs where you can interpolate between any two objects to produce infinitely many intermediate objects. Your prompt is a way to address a particular location of the database: if you ask, “How do you sort a list in Python? Answer like a pirate,” that’s a kind of database lookup, where you first retrieve a piece of knowledge (how to sort a list in Python) and then retrieve and execute a style transfer program (“Answer like a pirate”).

Since the knowledge and programs indexed by the LLM are interpolative, you can *move around in latent space* to explore nearby locations. A slightly different prompt, like “Explain Python list sorting, but answer like a buccaneer” would still have pointed to a very similar location in the database, resulting in an answer that would be pretty close but not quite identical. There are thousands of variations you could have used, each resulting in a similar yet slightly different answer. And that’s why prompt engineering is needed. There is no a priori reason for your first, naive prompt to be optimal for your task. The LLM is not going to understand what you meant and then perform it in the best possible way—it’s merely going to fetch the program that your prompt points to, among many possible locations you could have landed on.

Prompt engineering is the process of searching through latent space to find the lookup query that seems to perform best on your target task by trial and error. It’s no different from trying different keywords when doing a Google search. If LLMs actually understood what you asked them, there would be no need for this search process, since the amount of information conveyed about your target task does not change whether

¹ See also Marianna Nezhurina, Lucia Cipolina-Kun, Mehdi Cherti, and Jenia Jitsev, “Alice in Wonderland: Simple Tasks Showing Complete Reasoning Breakdown in State-Of-the-Art Large Language Models,” arXiv, <https://arxiv.org/abs/2406.02061>.

your prompt uses the word “rewrite” instead of “rephrase” or whether you prefix your prompt with “Think step by step.” Never assume that the LLM “gets it” the first time—keep in mind that your prompt is but an address in an infinite ocean of programs, all memorized as a by-product of learning to complete an enormous amount of token sequences.

19.1.3 Deep learning models struggle to learn generalizable programs

The problem with deep learning models isn’t just that they’re limited to blindly reapplying patterns they’ve memorized at training time or that they’re highly sensitive to how their input is presented. Even if you just need to query and apply a well-known program, and you know exactly how to address this program in latent space, you still face a major issue: the programs memorized by deep learning models often don’t generalize well. They will work for some input values and fail for some other input values. This is especially true for programs that encode any kind of discrete logic.

Consider the problem of adding two numbers, represented as character sequences—like “4 3 5 7 + 8 9 3 6.” Try training a Transformer on hundreds of thousands of such digit pairs: you will reach a very high accuracy. Very high, but not 100%—you will keep regularly seeing incorrect answers, because the Transformer doesn’t manage to encode the exact addition algorithm (you know, the one you learned in primary school). It is instead guessing the output by interpolating between the data points it has seen at training time.

This applies to state-of-the-art LLMs, too—at least those that weren’t explicitly hard-coded to execute snippets like “4357 + 8936” in Python to provide the right answer. They’ve seen enough examples of digit addition that they can add numbers, but they only have about 70% accuracy—quite underwhelming. Further, their accuracy is strongly dependent on *which* digits are being added, with more common digits leading to higher accuracy.

The reason why a deep learning model does not end up learning an exact addition algorithm even after seeing millions of examples is that it is just *a static chain of simple, continuous geometric transformations* mapping one vector space into another. That is a good fit for perceptual pattern recognition, but it’s a very poor fit for encoding any sort of step-by-step discrete logic, such as concepts like place value or carrying over. All it can do is map one data manifold X into another manifold Y , assuming the existence of a learnable continuous transform from X to Y . A deep learning model can be interpreted as a kind of program, but inversely, *most programs can’t be expressed as deep-learning models*. For most tasks, either there exists no corresponding neural network of reasonable size that solves the task or, even if one exists, it may not be *learnable*: the corresponding geometric transform may be far too complex, or there may not be appropriate data available to learn it.

19.1.4 The risk of anthropomorphizing machine-learning models

Our own understanding of images, sounds, and language is grounded in our sensorimotor experience as humans. Machine learning models have no access to such

experiences and thus can't understand their inputs in a human-relatable way. By feeding a large number of training examples into our models, we get them to learn a geometric transform that maps data to human concepts on a specific set of examples, but this mapping is a simplistic sketch of the original model in our minds—the one developed from our experience as embodied agents. It's like a dim image in a mirror (see figure 19.4). The models you create will take any shortcut available to fit their training data.

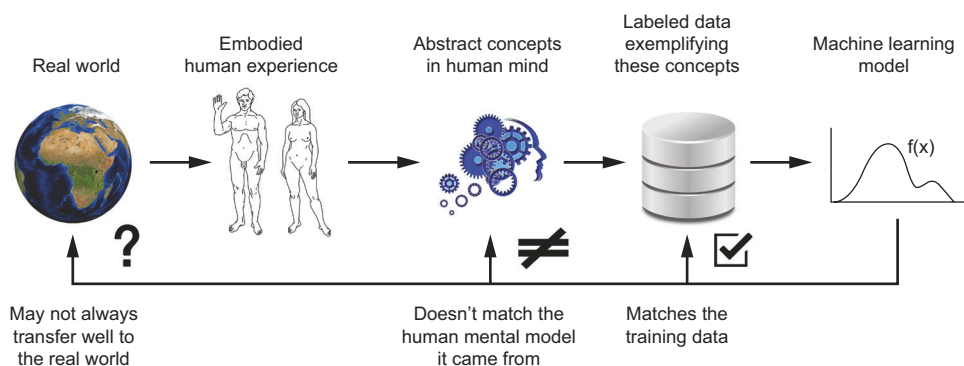


Figure 19.4 Current machine-learning models: like a dim image in a mirror

One real risk with contemporary AI is misinterpreting what deep learning models do and overestimating their abilities. A fundamental feature of humans is our *theory of mind*: our tendency to project intentions, beliefs, and knowledge on the things around us. Drawing a smiley face on a rock suddenly makes it “happy”—in our minds. Applied to deep learning, this means that when we train models capable of using language, we’re led to believe that the model “understands” the contents of the word sequences they generate just the way we do. Then we’re surprised when any slight departure from the patterns present in the training data causes the model to generate completely absurd answers.

As a machine learning practitioner, always be mindful of this and never fall into the trap of believing that neural networks understand the task they perform—they don’t, at least not in a way that would make sense to us. They were trained on a different, far narrower task than the one we wanted to teach them: that of mapping training inputs to training targets, point by point. Show them anything that deviates from their training data, and they will break in absurd ways.

19.2 Scale isn’t all you need

Could we just keep scaling our models to overcome the limitations of deep learning? Is *scale* all we need? This has long been the prevailing narrative in the field, one that was especially prominent in early 2023, during peak LLM hype. Back then, GPT-4 had just

been released, and it was essentially a scaled-up version of GPT-3: more parameters, more training data. Its significantly improved performance seemed to suggest that you could just keep going—that there could be a GPT-5 that would simply be more of the same and from which artificial general intelligence (AGI) would spontaneously emerge.

Proponents of this view would point to “scaling laws” as evidence. Scaling laws are an empirical relationship observed between the size of a deep learning model (as well as the size of its training dataset) and its performance on specific tasks. They suggest that increasing the size of a model reliably leads to better performance in a predictable manner. But the key thing that scaling law enthusiasts are missing is that the benchmarks they’re using to measure “performance” are effectively memorization tests, the kind we like to give university students. LLMs perform well on these tests by memorizing the answers, and naturally, cramming more questions and more answers into the models improves their performance accordingly.

The reality is that scaling up our models hasn’t led to any progress on the issues I’ve listed so far in these pages—inability to adapt to novelty, oversensitivity to phrasing, and the inability to infer generalizable programs for reasoning problems—because these issues are inherent to curve fitting, the paradigm of deep learning. I started pointing out these problems in 2017, and we’re still struggling with them today—with models that are now four or five orders of magnitude larger and more knowledgeable. We have not made any progress on these problems because *the models we’re using are still the same*. They’ve been the same for over seven years—they’re still parametric curves fitted to a dataset via gradient descent, and they’re still using the Transformer architecture.

Scaling up current deep learning techniques by stacking more layers and using more training data won’t solve the fundamental problems of deep learning:

- Deep-learning models are limited to using interpolative programs they memorize at training time. They are not able, on their own, to synthesize brand-new programs at inference time to adapt to substantially novel situations.
- Even within known situations, these interpolative programs suffer from generalization issues, which lead to oversensitivity to phrasing and confounder features.
- Deep learning models are limited in what they can represent, and most of the programs you may wish to learn can’t be expressed as a continuous geometric morphing of a data manifold. This is true in particular of algorithmic reasoning tasks.

Let’s take a closer look at what separates biological intelligence from the deep learning approach.

19.2.1 Automotons vs. intelligent agents

There are fundamental differences between the straightforward geometric morphing from input to output that deep learning models do and the way humans think and learn. It isn’t just the fact that humans learn by themselves from embodied experience instead of being presented with explicit training examples. The human brain is an entirely different beast compared to a differentiable parametric function.

Let's zoom out a little bit and ask, What's the purpose of intelligence? Why did it arise in the first place? We can only speculate, but we can make fairly informed speculations. We can start by looking at brains—the organ that produces intelligence. Brains are an evolutionary adaptation—a mechanism developed incrementally over hundreds of millions of years, via random trial and error guided by natural selection, that dramatically expanded the ability of organisms to adapt to their environment. Brains originally appeared more than half a billion years ago as a way to *store and execute behavioral programs*. Behavioral programs are just sets of instructions that make an organism reactive to its environment: “If this happens, then do that.” They link the organism's sensory inputs to its motor controls. In the beginning, brains would have served to hardcode behavioral programs (as neural connectivity patterns), which would allow an organism to react appropriately to its sensory input. This is the way insect brains still work—flies, ants, *C. elegans* (see figure 19.5), etc. Because the original “source code” of these programs was DNA, which would get decoded as neural connectivity patterns, evolution was suddenly able to *search over behavior space* in a largely unbounded way—a major evolutionary shift.

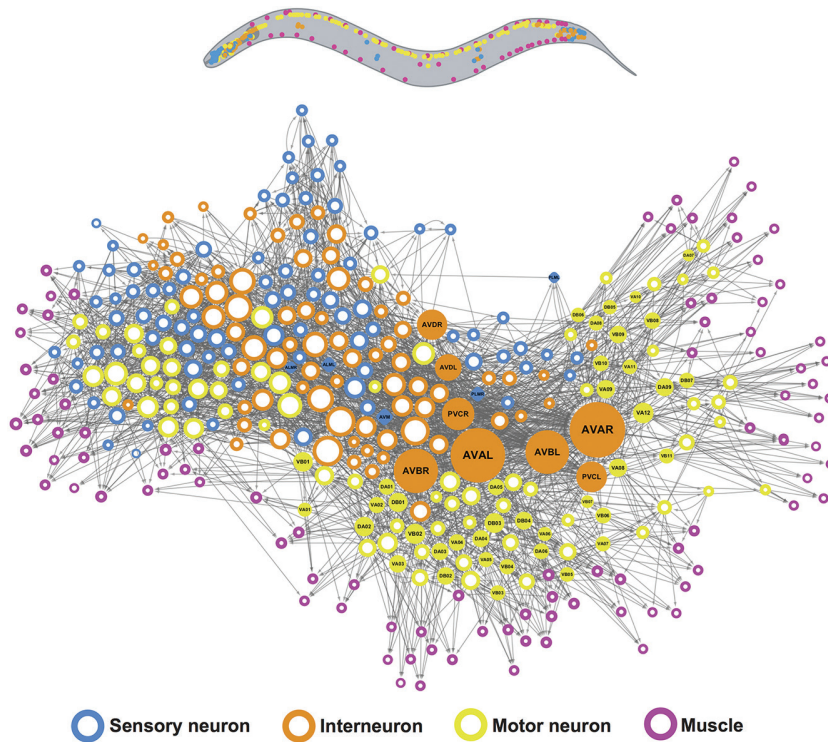


Figure 19.5 The brain network of the *C. elegans* worm: a behavioral automaton “programmed” by natural evolution. Figure created by Emma Towilson (from “Network control principles predict neuron function in the *Caenorhabditis elegans* connectome,” Yan et al., *Nature*, Oct. 2017).

Evolution was the programmer, and brains were computers carefully executing the code evolution gave them. Because neural connectivity is a very general computing substrate, the sensorimotor space of all brain-enabled species could suddenly start undergoing a dramatic expansion. Eyes, ears, mandibles, 4 legs, 24 legs—as long as you have a brain, evolution will kindly figure out for you behavioral programs that make good use of these. Brains can handle any modality, or combination of modalities, you throw at them.

Now, mind you, these early brains weren't exactly intelligent per se. They were very much *automatons*: they would merely execute behavioral programs hardcoded in the organism's DNA. They could only be described as intelligent in the same sense that a thermostat is “intelligent.” Or a list-sorting program. Or a trained deep neural network (of the artificial kind). This is an important distinction, so let's look at it carefully: What's the difference between automatons and actual intelligent agents?

19.2.2 Local generalization vs. extreme generalization

The field of AI has long suffered from conflating the notions of *intelligence* and *automation*. An automation system (or automaton) is static, crafted to accomplish specific things in a specific context—“If this, then that”—while an intelligent agent can adapt on the fly to novel, unexpected situations. When an automaton is exposed to something that doesn't match what it was “programmed” to do (whether we're talking about human-written programs, evolution-generated programs, or the implicit programming process of fitting a model on a training dataset), it will fail.

Meanwhile, intelligent agents, like us humans, will use their fluid intelligence to find a way forward. How do you tell the difference between a student who has memorized the past three years of exam questions but has no understanding of the subject and one who actually understands the material? You give them a brand-new problem.

Humans are capable of far more than mapping immediate stimuli to immediate responses as a deep network or an insect would. We can assemble on-the-fly complex, abstract models of our current situation, of ourselves, and of other people, and can use these models to anticipate different possible futures and perform long-term planning. We can quickly adapt to unexpected situations and pick up new skills after just a little bit of practice.

This ability to use *abstraction* and *reasoning* to handle experiences we weren't prepared for is the defining characteristic of human cognition. I call it *extreme generalization*: an ability to adapt to novel, never-before-experienced situations using little data or even no new data at all. This capability is key to the intelligence displayed by humans and advanced animals.

This stands in sharp contrast with what automaton-like systems do. A very rigid automaton wouldn't feature any generalization at all; it would be incapable of handling anything that it wasn't precisely told about in advance. A Python dict, or a basic question-answering program implemented as hardcoded if-then-else statements would fall into this category. Deep nets do slightly better: they can successfully process

inputs that deviate a bit from what they're familiar with, which is precisely what makes them useful. Our dogs-versus-cats model from chapter 8 could classify cat or dog pictures it had not seen before, as long as they were close enough to what it was trained on. However, deep nets are limited to what I call *local generalization* (see figure 19.6): the mapping from inputs to outputs performed by a deep net quickly stops making sense as inputs start deviating from what the net saw at training time. Deep nets can only generalize to *known unknowns*, to factors of variation that were anticipated during model development and that are extensively featured in the training data, such as different camera angles or lighting conditions for pet pictures. That's because deep nets generalize via interpolation on a manifold (remember chapter 5): any factor of variation in their input space needs to be captured by the manifold they learn. That's why basic data augmentation is so helpful in improving deep net generalization. Unlike humans, these models have no ability to improvise in the face of situations for which little or no data is available.

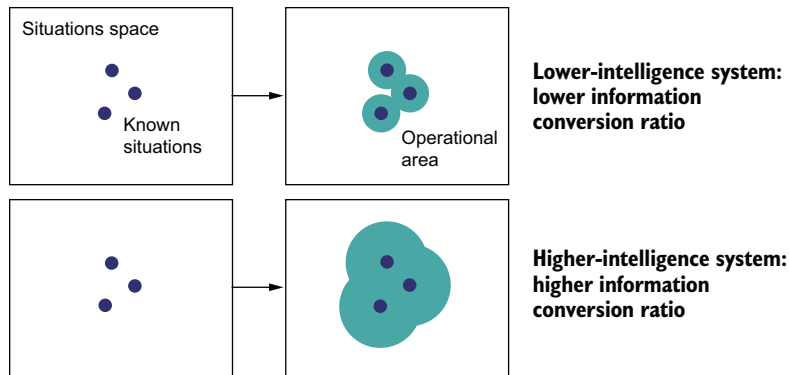


Figure 19.6 Local generalization vs. extreme generalization

Consider, for instance, the problem of learning the appropriate launch parameters to get a rocket to land on the moon. If you used a deep net for this task and trained it using supervised learning or reinforcement learning, you'd have to feed it tens of thousands or even millions of launch trials: you'd need to expose it to a *dense sampling* of the input space for it to learn a reliable mapping from input space to output space. In contrast, as humans, we can use our power of abstraction to come up with physical models—rocket science—and derive an exact solution that will land the rocket on the moon in one or a few trials. Similarly, if you developed a deep net controlling a human body and you wanted it to learn to safely navigate a city without getting hit by cars, the net would have to die many thousands of times in various situations until it could infer that cars are dangerous and develop appropriate avoidance behaviors. Dropped into a new city, the net would have to relearn most of what it knows. On the other

hand, humans are able to learn safe behaviors without having to die even once—again, thanks to our power of abstract modeling of novel situations.

19.2.3 *The purpose of intelligence*

This distinction between highly adaptable intelligent agents and rigid automatons leads us back to brain evolution. Why did brains—originally a mere medium for natural evolution to develop behavioral automatons—eventually turn intelligent? Like every significant evolutionary milestone, it happened because natural selection constraints encouraged it to happen.

Brains are responsible for behavior generation. If the set of situations an organism had to face was mostly static and known in advance, behavior generation would be an easy problem: evolution would just figure out the correct behaviors via random trial and error and hardcode them into the organism's DNA. This first stage of brain evolution—brains as automatons—would already be optimal. However, crucially, as organism complexity—and alongside it, environmental complexity—kept increasing, the situations animals had to deal with became much more dynamic and more unpredictable. A day in your life, if you look closely, is unlike any day you've ever experienced and unlike any day ever experienced by any of your evolutionary ancestors. You need to be able to face unknown and surprising situations constantly. There is no way for evolution to find and hardcode as DNA the sequence of behaviors you've been executing to successfully navigate your day since you woke up a few hours ago. It has to be generated on the fly every day.

The brain, as a good behavior-generation engine, simply adapted to fit this need. It optimized for adaptability and generality themselves, rather than merely optimizing for fitness to a fixed set of situations. This shift likely occurred multiple times throughout evolutionary history, resulting in highly intelligent animals in very distant evolutionary branches—apes, octopuses, ravens, and more. Intelligence is an answer to challenges presented by complex, dynamic ecosystems.

That's the nature of intelligence: it is the ability to efficiently use the information at your disposal to produce successful behavior in the face of an uncertain, ever-changing future. What Descartes calls “understanding” is the key to this remarkable capability: the power to mine your past experience to develop modular, reusable abstractions that can be quickly repurposed to handle novel situations and achieve extreme generalization.

19.2.4 *Climbing the spectrum of generalization*

As a crude caricature, you could summarize the evolutionary history of biological intelligence as a slow climb up the *spectrum of generalization*. It started with automaton-like brains that could only perform local generalization. Over time, evolution started producing organisms capable of increasingly broader generalization that could thrive in ever-more complex and variable environments. Eventually, in the past few million years—an instant in evolutionary terms—certain hominin species started trending toward an implementation of biological intelligence capable of extreme

generalization, precipitating the start of the Anthropocene and forever changing the history of life on Earth.

The progress of AI over the past 70 years bears striking similarities to this evolution. Early AI systems were pure automatons, like the ELIZA chat program from the 1960s, or SHRDLU,² a 1970 AI capable of manipulating simple objects from natural language commands. In the 1990s and 2000s, we saw the rise of machine learning systems capable of local generalization that could deal with some level of uncertainty and novelty. In the 2010s, deep learning further expanded the local generalization power of these systems by enabling engineers to use much larger datasets and much more expressive models.

Today, we may be on the cusp of the next evolutionary step. We are moving toward systems that achieve *broad generalization*, which I define as the ability to deal with *unknown unknowns* within a single broad domain of tasks (including situations the system was not trained to handle and that its creators could not have anticipated). Examples are a self-driving car capable of safely dealing with any situation you throw at it or a domestic robot that could pass the “Woz test of intelligence”—entering a random kitchen and making a cup of coffee.³ By combining deep learning and painstakingly handcrafted abstract models of the world, we’re already making visible progress toward these goals.

However, the deep learning paradigm has remained limited to cognitive automation: The “intelligence” label in “artificial intelligence” has been a category error. It would be more accurate to call our field “artificial cognition,” with “cognitive automation” and “artificial intelligence” being two nearly independent subfields within it. In this subdivision, AI would be a greenfield where almost everything remains to be discovered.

Now, I don’t mean to diminish the achievements of deep learning. Cognitive automation is incredibly useful, and the way deep learning models are capable of automating tasks from exposure to data alone represents an especially powerful form of cognitive automation, far more practical and versatile than explicit programming. Doing this well is a game changer for essentially every industry. But it’s still a long way from human (or animal) intelligence. Our models, so far, can only perform local generalization: they map space X to space Y via a smooth geometric transform learned from a dense sampling of X-to-Y data points, and any disruption within spaces X or Y invalidates this mapping. They can only generalize to new situations that stay similar to past data, whereas human cognition is capable of extreme generalization, quickly adapting to radically novel situations and planning for long-term future situations.

19.3 *How to build intelligence*

So far, you’ve learned that there’s a lot more to intelligence than the sort of latent manifold interpolation that deep learning does. But what, then, do we need to start building real intelligence? What are the core pieces that are currently eluding us?

² Terry Winograd, “Procedures as a Representation for Data in a Computer Program for Understanding Natural Language,” 1971.

³ Michael Shick, “Wozniak: Could a Computer Make a Cup of Coffee?” Fast Company, March 2010.

19.3.1 The kaleidoscope hypothesis

Intelligence is the ability to use your past experience (and innate prior knowledge) to face novel, unexpected future situations. Now, if the future you had to face was *truly novel*—sharing no common ground with anything you’ve seen before—you’d be unable to react to it, no matter how intelligent you are.

Intelligence works because nothing is ever truly without precedent. When we encounter something new, we’re able to make sense of it by drawing analogies to our past experience and articulating it in terms of the abstract concepts we’ve collected over time. A person from the 17th century seeing a jet plane for the first time might describe it as a large, loud metal bird that doesn’t flap its wings. A car? That’s a horseless carriage. If you’re trying to teach physics to a grade schooler, you can explain how electricity is like water in a pipe or how spacetime is like a rubber sheet getting distorted by heavy objects.

Besides such clear-cut, explicit analogies, we’re constantly making smaller, implicit analogies—every second, with every thought. Analogies are how we navigate life. Shopping at a new supermarket? You’ll find your way by relating it to similar stores you’ve been to. Talking to someone new? They’ll remind you of a few people you’ve met before. Even seemingly random patterns, like the shape of clouds, instantly evoke in us vivid images—an elephant, a ship, a fish.

These analogies aren’t just in our minds, either: physical reality itself is full of isomorphisms. Electromagnetism is analogous to gravity. Animals are all structurally similar to each other, due to shared origins. Silica crystals are similar to ice crystals. And so on.

I call this the *kaleidoscope hypothesis*: our experience of the world seems to feature incredible complexity and never-ending novelty, but everything in this sea of complexity is similar to everything else. The number of *unique atoms of meaning* that you need to describe the universe you live in is relatively small, and everything around you is a recombination of these atoms: a few seeds, endless variation, much like what goes on inside a kaleidoscope, where a few glass beads are reflected by a system of mirrors to produce rich, seemingly endless patterns (see figure 19.7).

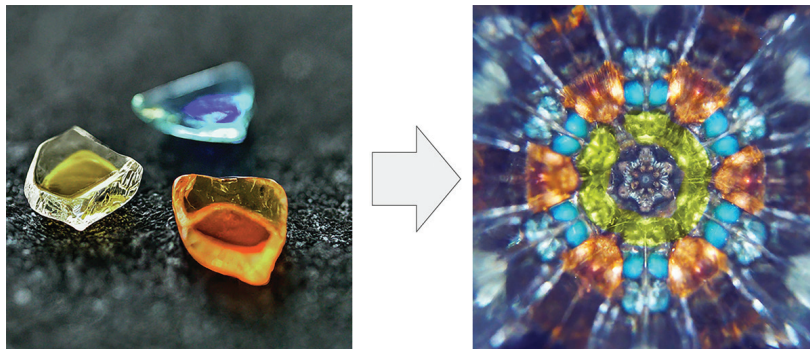


Figure 19.7 A kaleidoscope produces rich (yet repetitive) patterns from just a few beads of colored glass.

19.3.2 The essence of intelligence: Abstraction acquisition and recombination

Intelligence is the ability to mine your experience to identify these atoms of meaning that can seemingly be reused across many different situations—the core beads of the kaleidoscope. Once extracted, they’re called *abstractions*. Whenever you encounter a new situation, you make sense of it by recombining on the fly abstractions from your collections, to weave a brand new “model” adapted to the situation.

This process consists of two key parts:

- *Abstraction acquisition*—Efficiently extracting compact, reusable abstractions from a stream of experience or data. This involves identifying underlying structures, principles, or invariants.
- *On-the-fly recombination*—Efficiently selecting and recombining these abstractions in novel ways to model new problems and situations, even ones far removed from past experience.

The emphasis on *efficiency* is crucial. How intelligent you are is determined by how efficiently you can acquire good abstractions from limited experience and how efficiently you can recombine them to navigate uncertainty and novelty. If you need hundreds of thousands of hours of practice to acquire a skill, you are not very intelligent. If you need to enumerate every possible move on the chess board to find the best one, you are not very intelligent.

And that’s the source of the two main issues with the classic deep learning paradigm:

- These models are completely missing on-the-fly recombination. They do a decent job at acquiring abstractions at training time, via gradient descent, but by design they have zero ability to recombine what they know at test time. They behave like a static abstract database, limited purely to retrieval. They’re missing half of the picture—the most important half.
- They’re terribly inefficient. Gradient descent requires vast amounts of data to distill neat abstractions—many orders of magnitude more data than humans.

So how can we move beyond these limitations?

19.3.3 The importance of setting the right target

Biological intelligence was the answer to a question asked by nature. Likewise, if we want to develop true AI, first, we need to be asking the right questions. Ultimately, the capabilities of AI systems reflect the objectives they were designed and optimized for.

An effect you see constantly in systems design is the *shortcut rule*: if you focus on optimizing one success metric, you will achieve your goal, but at the expense of everything in the system that wasn’t covered by your success metric. You end up taking every available shortcut toward the goal. Your creations are shaped by the incentives you give yourself.

You see this often in machine learning competitions. In 2009, Netflix ran a challenge that promised a \$1 million prize to the team that would achieve the highest score on a movie recommendation task. It ended up never using the system created by the winning team because it was way too complex and compute intensive. The winners had

optimized for prediction accuracy alone—what they were incentivized to achieve—at the expense of every other desirable characteristic of the system: inference cost, maintainability, explainability. The shortcut rule holds true in most Kaggle competitions as well—the models produced by Kaggle winners can rarely, if ever, be used in production.

The shortcut rule has been everywhere in AI over the past few decades. In the 1970s, psychologist and computer science pioneer Allen Newell, concerned that his field wasn't making any meaningful progress toward a proper theory of cognition, proposed a new grand goal for AI: chess playing. The rationale was that playing chess, in humans, seemed to involve—perhaps even require—capabilities such as perception, reasoning and analysis, memory and study from books, and so on. Surely, if we could build a chess-playing machine, it would have to feature these attributes as well. Right?

Over two decades later, the dream came true: in 1997, IBM's Deep Blue beat Gary Kasparov, the best chess player in the world. Researchers had then to contend with the fact that creating a chess-champion AI had taught them little about human intelligence. The A-star algorithm at the heart of Deep Blue wasn't a model of the human brain and couldn't generalize to tasks other than similar board games. It turned out it was easier to build an AI that could only play chess than to build an artificial mind—so that's the shortcut researchers took.

So far, *the driving success metric of the field of AI has been to solve specific tasks*, from chess to Go, from MNIST classification to ImageNet, from high school math tests to the bar exam. Consequently, the history of the field has been defined by a series of “successes” where *we figured out how to solve these tasks without featuring any intelligence*.

If that sounds like a surprising statement, keep in mind that human-like intelligence isn't characterized by skill at any particular task—rather, it is the ability to adapt to novelty to efficiently acquire new skills and master never-before-seen tasks. By fixing the task, you make it possible to provide an arbitrarily precise description of what needs to be done—either via hardcoding human-provided knowledge or by supplying humongous amounts of data. You make it possible for engineers to “buy” more skill for their AI by just adding data or adding hardcoded knowledge, without increasing the generalization power of the AI (see figure 19.8). If you have near-infinite training data, even a very crude algorithm like nearest-neighbor search can play video games with superhuman skill. Likewise, if you have a near-infinite amount of human-written if-then-else statements—that is, until you make a small change to the rules of the game, the kind a human could adapt to instantly—that will require the unintelligent system to be retrained or rebuilt from scratch.

In short, by fixing the task, you remove the need to handle uncertainty and novelty, and since the nature of intelligence is the ability to handle uncertainty and novelty, you're effectively removing the need for intelligence. And because it's always easier to find an unintelligent solution to a specific task than to solve the general problem of intelligence, that's the shortcut you will take 100% of the time. Humans can use their general intelligence to acquire skills at any new task, but in reverse, there is no path from a collection of task-specific skills to general intelligence.

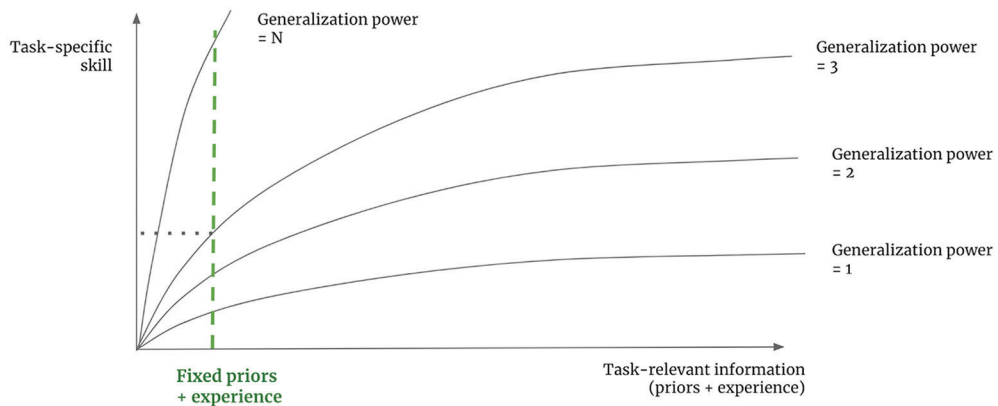


Figure 19.8 A low-generalization system can achieve arbitrary skill at a fixed task given unlimited task-specific information.

19.3.4 A new target: On-the-fly adaptation

To make AI actually intelligent and give it the ability to deal with the incredible variability and ever-changing nature of the real world, first, we need to move away from seeking to achieve *task-specific skill* and, instead, start targeting generalization power itself. We need new metrics of progress that will help us develop increasingly intelligent systems: metrics that will point in the right direction and that will give us an actionable feedback signal. As long as we set our goal to be “create a model that solves task X,” the shortcut rule will apply, and we’ll end up with a model that does X, period.

In my view, intelligence can be precisely quantified as an *efficiency ratio*: the conversion ratio between the *amount of relevant information* you have available about the world (which could be either past experience or innate prior knowledge) and your *future operating area*, the set of novel situations where you will be able to produce appropriate behavior (you can view this as your *skill set*). A more intelligent agent will be able to handle a broader set of future tasks and situations using a smaller amount of past experience. To measure such a ratio, you just need to fix the information available to your system—its experience and its prior knowledge—and measure its performance on a set of reference situations or tasks that are known to be sufficiently different from what the system has had access to. Trying to maximize this ratio should lead you toward intelligence. Crucially, to avoid cheating, you’re going to need to make sure to test the system only on tasks it wasn’t programmed or trained to handle—in fact, you need tasks that the *creators of the system could not have anticipated*.

In 2018 and 2019, I developed a benchmark dataset called the *Abstraction & Reasoning Corpus for Artificial General Intelligence (ARC-AGI)*⁴ that seeks to capture this definition

⁴ François Chollet, “On the Measure of Intelligence,” 2019.

of intelligence. ARC-AGI is meant to be approachable by both machines and humans, and it looks very similar to human IQ tests, such as Raven’s progressive matrices. At test time, you’ll see a series of “tasks.” Each task is explained via three or four “examples” that take the form of an input grid and a corresponding output grid (see figure 19.9). You’ll then be given a brand-new input grid, and you’ll have three tries to produce the correct output grid, before moving on to the next task.

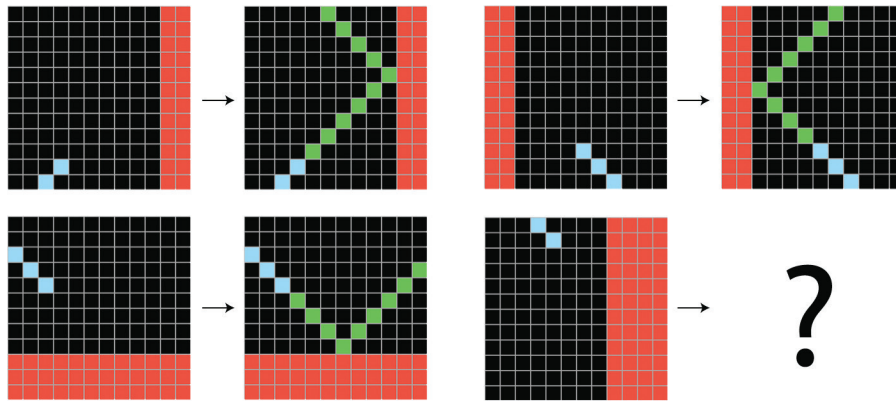


Figure 19.9 An ARC-AGI task: the nature of the task is demonstrated by a couple of input-output pair examples. Provided with a new input, you must construct the corresponding output.

Compared to IQ tests, two things are unique about ARC-AGI. First, ARC seeks to measure generalization power by only testing you on tasks you’ve never seen before. That means that ARC-AGI is *a game you can’t practice for*, at least in theory: the tasks you will get tested on will have their own unique logic that you will have to understand on the fly. You can’t just memorize specific strategies from past tasks.

In addition, ARC-AGI tries to control for the *prior knowledge* that you bring to the test. You never approach a new problem entirely from scratch—you bring to it preexisting skills and information. ARC-AGI makes the assumption that all test takers should start from the set of knowledge priors, called *Core Knowledge priors*, which represent the knowledge systems humans are born with. Unlike an IQ test, ARC-AGI tasks will never involve acquired knowledge, like English sentences, for instance.

19.3.5 ARC Prize

In 2024, to accelerate progress toward AI systems capable of the kind of fluid abstraction and reasoning measured by ARC-AGI, I partnered with Mike Knoop to establish the nonprofit ARC Prize Foundation. The foundation runs a yearly competition, with a substantial prize pool (over \$1 million in its 2024 iteration) to incentivize researchers to develop AI that can solve ARC-AGI and thus display genuine fluid intelligence.

The ARC-AGI benchmark has proven remarkably resistant to the prevailing deep learning scaling paradigm. Most other benchmarks have saturated quickly in the age of LLMs. That's because they can be hacked via memorization, whereas ARC-AGI is designed to be resistant to it. From 2019, when ARC-AGI was first released, to 2025, base LLMs underwent a roughly 50,000× scale-up—from, say, GPT-2 (2019) to GPT-4.5 (2025), but their performance on the 2019 version of ARC-AGI only went from 0% to around 10%. Given that you, reader, would easily score above 95%, that isn't very good.

If you scale up your system by 50,000× and you're still not making meaningful progress, that's like a big warning sign telling you that you need to try new ideas. Simply making models bigger or training them on more data has not unlocked the kind of fluid intelligence that ARC-AGI requires. ARC-AGI was clearly showing that on-the-fly recombination capabilities are necessary to tackle reasoning.

19.3.6 *The test-time adaptation era*

In 2024, everything changed. That year saw a major narrative shift—one that was partly catalyzed by ARC Prize. The prevailing “Scale is all you need” story that was a bedrock dogma of 2023 started giving way to “Actually, we need on-the-fly recombination.” The results of the competition, announced in December 2024, were illuminating: the leading solutions did not emerge from simply scaling existing deep learning architectures. They all used some form of test-time adaptation (TTA)—either test-time search or test-time training.

TTA refers to methods where the AI system performs active reasoning or learning during the test itself, using the specific problem information provided—the key component that was missing from the classic deep learning paradigm.

There are several ways to implement test-time adaptation:

- *Test-time training*—The model adjusts some of its parameters based on the examples given in the test task, using gradient descent.
- *Search methods*—The system searches through many possible reasoning steps or potential solutions at test time to find the best one. This could be done in natural language (chain-of-thought synthesis) or in a space of symbolic, verifiable programs (program synthesis).

These TTA approaches allow AI systems to be more flexible and handle novelty better than static models. Every single top entry in ARC Prize 2024 used them.

Shortly following the competition's conclusion, in late December 2024, OpenAI previewed its o3 test-time reasoning model and used ARC-AGI to showcase its unprecedented capabilities. Using considerable test-time compute resources, this model achieved scores of 76% at a cost of about \$200 per task, and 88% at a cost of over \$20,000 per task, surpassing the nominal human baseline. For the very first time, we were seeing an AI model that showed signs of genuine fluid intelligence. This breakthrough opened the floodgates of a new wave of interest and investment in similar techniques—the test-time adaptation era had begun. Importantly, ARC-AGI was one of the only benchmarks at the time that provided a clear signal that a major paradigm shift was underway.

19.3.7 ARC-AGI 2

Does that mean AGI is solved? Was o3 as intelligent as a human?

Not quite. First, while o3's performance was a landmark achievement, it came at a tremendous cost—tens of thousands of dollars of compute per ARC-AGI puzzle. Intelligence isn't just about capability; it's fundamentally about efficiency. Brute-forcing the solution space given enormous compute is a shortcut that makes all kinds of tasks possible without requiring intelligence. In principle, you could even solve ARC-AGI by simply walking down the tree of every possible solution program and testing each one until you find one that works on the demonstration pairs. The o3 results, impressive as they were, felt more like cracking a code with a supercomputer than a display of nimble, human-like fluid reasoning. The entire point of intelligence is to achieve results with the least amount of resources possible.

Second, we found that o3 was still stumped by many tasks that humans found very easy (like the one in figure 19.10). This strongly suggests that o3 wasn't quite human-level yet. Here's the thing—the 2019 version of ARC-AGI was intended to be easy. It was essentially a binary test of fluid intelligence: either you have no fluid intelligence, like all base LLMs, in which case you score near zero, or you do display some genuine fluid intelligence, in which case you immediately score extremely high, like any human—or o3. There wasn't much room in between. It was clear that the benchmark needed to evolve alongside the AI capabilities it was designed to measure. There was a need for a new ARC-AGI version that was less brute-forcible and that could better differentiate between systems possessing varying levels of fluid reasoning ability, up to human-level fluid intelligence. Good news: we had been working on one since 2022.

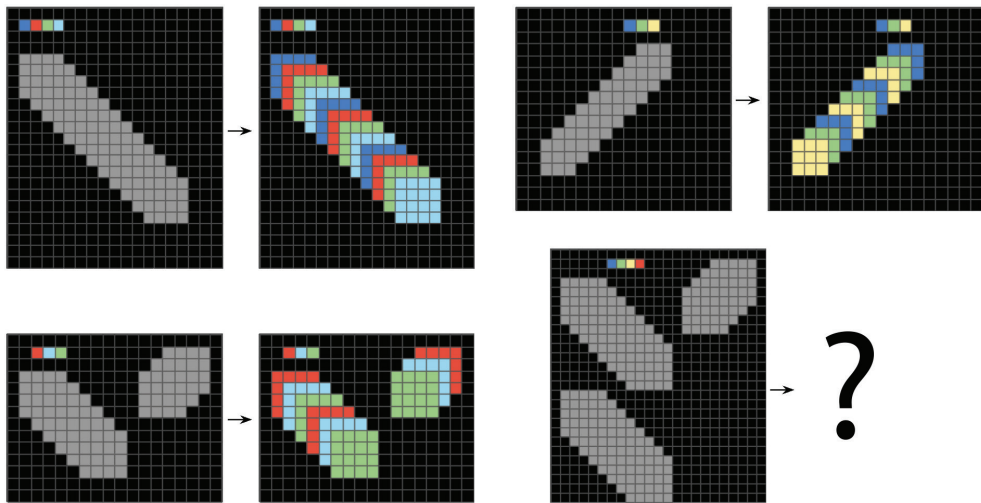


Figure 19.10 Example of a task that couldn't be solved by o3 on the highest compute settings (over \$20,000 per task)

And so, in March 2025, the ARC Prize Foundation introduced ARC-AGI-2. It kept the exact same format as the first version but significantly improved the task content. The new iteration was designed to raise the bar, incorporating tasks that demand more complex reasoning chains and are inherently more resistant to exhaustive search methods. The goal was to create a benchmark where computational efficiency becomes a more critical factor for success, pushing systems toward more genuinely intelligent, efficient strategies rather than simply exploring billions of possibilities. While most ARC-AGI-1 tasks could be solved almost instantaneously by a human without requiring much cognitive effort, all tasks in ARC-AGI-2 require some amount of deliberate thinking (see figure 19.11)—for instance, the average time for task completion among human test takers in our experiments was 5 minutes.

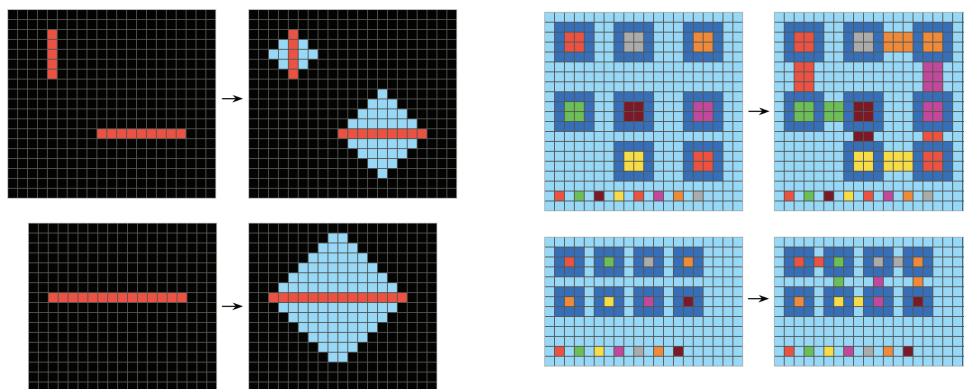


Figure 19.11 Typical ARC-AGI-1 task (left) vs. typical ARC-AGI-2 task (right)

The initial AI testing results on ARC-AGI 2 were sobering: even o3 struggled significantly with this new set of challenges, its scores plummeting back into the low double digits when constrained to reasonable computational budgets. As for base LLMs? Their performance on ARC-AGI-2 was effectively back at 0%—fitting, as base LLMs don't possess fluid intelligence. The challenge of building AI with truly efficient, human-like fluid intelligence is still far from solved. We're going to need something beyond current TTA techniques.

19.4 *The missing ingredients: Search and symbols*

What would it take to fully solve ARC-AGI, in particular version 2? Hopefully, this challenge will get you thinking. That's the entire point of ARC-AGI: to give you a goal of a different kind, that will nudge you in a new direction—hopefully, a productive direction. Now, let's take a quick look at the key ingredients you're going to need if you want to answer the call.

I've said that intelligence consists of two components: *abstraction acquisition* and *abstraction recombination*. They are tightly coupled—what *kind* of abstractions you manipulate determines how, and how well, you can recombine them. Deep learning models only manipulate abstractions stored via parametric curves, fitted via gradient descent. Could there be a better way?

19.4.1 The two poles of abstraction

Abstraction acquisition starts with *comparing things to one another*. Crucially, there are two distinct ways to compare things, from which arise two different kinds of abstraction and two modes of thinking, each better suited to a different kind of problem. Together, these two poles of abstraction form the basis for all of our thoughts.

The first way to relate things to each other is *similarity comparison*, which gives rise to *value-centric analogies*. The second way is *exact structural match*, which gives rise to *program-centric analogies* (or structure-centric analogies). In both cases, you start from *instances* of a thing, and you merge together related instances to produce an *abstraction* that captures the common elements of the underlying instances. What varies is how you tell that two instances are related and how you merge instances into abstractions. Let's take a close look at each type.

VALUE-CENTRIC ANALOGY

Let's say you come across a number of different beetles in your backyard, belonging to multiple species. You'll notice similarities between them. Some will be more similar to one another, and some will be less similar: the notion of similarity is implicitly a smooth, continuous *distance function* that defines a latent manifold where your instances live. Once you've seen enough beetles, you can start clustering more similar instances together and merging them into a set of *prototypes* that captures the shared visual features of each cluster (figure 19.12). These prototypes are abstract: they don't look like any specific instance you've seen, although they encode properties that are common across all of them. When you encounter a new beetle, you won't need to compare it to every single beetle you've seen before to know what to do with it. You can simply compare it to your handful of prototypes to find the closest prototype—the beetle's *category*—and use it to make useful predictions: Is the beetle likely to bite you? Will it eat your apples?

Does this sound familiar? It's pretty much a description of what unsupervised machine learning (such as the K-means clustering algorithm) does. In general, all of modern machine learning, unsupervised or not, works by learning latent manifolds that describe a space of instances, encoded via prototypes. (Remember the ConvNet features you visualized in chapter 10? They were visual prototypes.) Value-centric analogy is the kind of analogy-making that enables deep learning models to perform local generalization.

It's also what many of your own cognitive abilities run on. As a human, you perform value-centric analogies all the time. It's the type of abstraction that underlies *pattern*

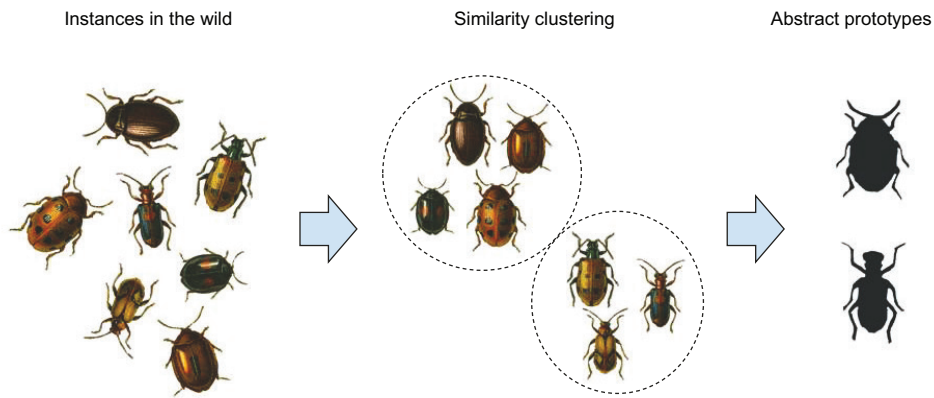


Figure 19.12 Value-centric analogy relates instances via a continuous notion of similarity to obtain abstract prototypes.

recognition, perception, and intuition. If you can do a task without thinking about it, you’re heavily relying on value-centric analogies. If you’re watching a movie and you start subconsciously categorizing the different characters into “types,” that’s value-centric abstraction.

PROGRAM-CENTRIC ANALOGY

Crucially, there’s more to cognition than the kind of immediate, approximate, intuitive categorization that value-centric analogy enables. There’s another type of abstraction-generation mechanism, slower, exact, deliberate: program-centric (or structure-centric) analogy.

In software engineering, you often write different functions or classes that seem to have a lot in common. When you notice these redundancies, you start asking, Could there be a more abstract function that performs the same job that could be reused twice? Could there be an abstract base class that both of your classes could inherit from? The definition of abstraction you’re using here corresponds to program-centric analogy. You’re not trying to compare your classes and functions by *how similar* they look, the way you’d compare two human faces, via an implicit distance function. Rather, you’re interested in whether there are *parts* of them that have *exactly the same structure*. You’re looking for what is called a *subgraph isomorphism* (see figure 19.13): programs can be represented as graphs of operators, and you’re trying to find subgraphs (program subsets) that are exactly shared across your different programs.

This kind of analogy-making via exact structural match within different discrete structures isn’t at all exclusive to specialized fields like computer science, or mathematics—you’re constantly using it without noticing. It underlies *reasoning, planning*, and the general concept of *rigor* (as opposed to intuition). Any time you’re thinking about objects connected to each other by a discrete network of relationships (rather than a continuous similarity function), you’re using program-centric analogies.

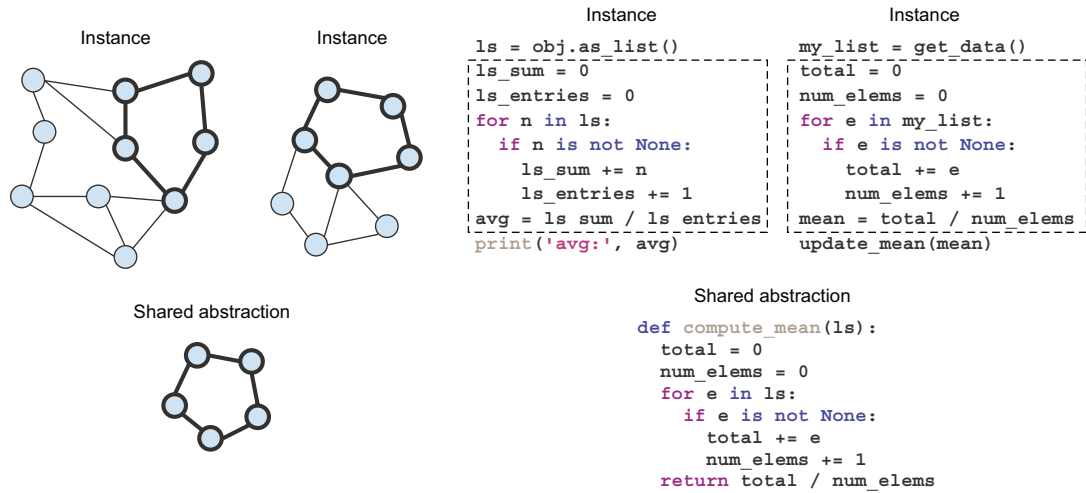


Figure 19.13 Program-centric analogy identifies and isolates isomorphic substructures across different instances.

19.4.2 Cognition as a combination of both kinds of abstraction

Table 19.1 compares these two poles of abstraction side by side.

Table 19.1 The two poles of abstraction

Value-centric abstraction	Program-centric abstraction
Relates things by distance	Relates things by exact structural match
Continuous, grounded in geometry	Discrete, grounded in topology
Produces abstractions by “averaging” instances into “prototypes”	Produces abstractions by isolating isomorphic substructures across instances
Underlies perception and intuition	Underlies reasoning and planning
Immediate, fuzzy, approximative	Slow, exact, rigorous
Requires a lot of experience to produce reliable results	Experience efficient: can operate on as few as two instances

Everything we do, everything we think, is a combination of these two types of abstraction. You’d be hard pressed to find tasks that only involve one of the two. Even a seemingly “pure perception” task, like recognizing objects in a scene, involves a fair amount of implicit reasoning about the relationships between the objects you’re looking at. And even a seemingly “pure reasoning” task, like finding the proof of a mathematical theorem, involves a good amount of intuition. When a mathematician puts their pen to the paper, they’ve already got a fuzzy vision of the direction in which they’re going. The discrete reasoning steps they take to get to the destination are guided by high-level intuition.

These two poles are complementary, and it's their interleaving that enables extreme generalization. No mind could be complete without both of them.

19.4.3 *Why deep learning isn't a complete answer to abstraction generation*

Deep learning is very good at encoding value-centric abstraction, but it has basically no ability to generate program-centric abstraction. Human-like intelligence is a tight interleaving of both types, so we're literally missing half of what we need—arguably the most important half.

Now, here's a caveat. So far, I've presented each type of abstraction as entirely separate from the other—opposite, even. In practice, however, they're more of a spectrum: to an extent, you could do reasoning by embedding discrete programs in continuous manifolds—just like you may fit a polynomial function through any set of discrete points, as long as you have enough coefficients. And inversely, you could use discrete programs to emulate continuous distance functions—after all, when you're doing linear algebra on a computer, you're working with continuous spaces, entirely via discrete programs that operate on 1s and 0s.

However, there are clearly types of problems that are better suited to one or the other. Try to train a deep learning model to sort a list of five numbers, for instance. With the right architecture, it's not impossible, but it's an exercise in frustration. You'll need a massive amount of training data to make it happen—and even then, the model will still make occasional mistakes when presented with new numbers. And if you want to start sorting lists of 10 numbers instead, you'll need to completely retrain the model—on even more data. Meanwhile, writing a sorting algorithm in Python takes just a few lines, and the resulting program, once validated on a couple more examples, will work every time on lists of any size. That's pretty strong generalization: going from a couple of demonstration examples and test examples to a program that can successfully process literally any list of numbers.

In reverse, perception problems are a terrible fit for discrete reasoning processes. Try to write a pure-Python program to classify MNIST digits, without using any machine learning technique: you're in for a ride. You'll find yourself painstakingly coding functions that can detect the number of closed loops in a digit, the coordinates of the center of mass of a digit, and so on. After thousands of lines of code, you might achieve 90% test accuracy. In this case, fitting a parametric model is much simpler; it can better utilize the large amount of data that's available, and it achieves much more robust results. If you have lots of data and you're faced with a problem where the manifold hypothesis applies, go with deep learning.

For this reason, it's unlikely that we'll see the rise of an approach that would reduce reasoning problems to manifold interpolation or that would reduce perception problems to discrete reasoning. The way forward in AI is to develop a unified framework that incorporates *both* types of abstraction generation.

19.4.4 An alternative approach to AI: Program synthesis

Until 2024, AI systems capable of genuine discrete reasoning were all hardcoded by human programmers—for instance, software that relies on search algorithms, graph manipulation, and formal logic. In the test-time adaptation (TTA) era, this is finally starting to change. A branch of TTA that is especially promising is *program synthesis*—a field that is still very niche today, but that I expect to take off in a big way over the next few decades.

Program synthesis consists of automatically generating simple programs by using a search algorithm (possibly genetic search, as in *genetic programming*) to explore a large space of possible programs (see figure 19.14). The search stops when a program is found that matches the required specifications, often provided as a set of input-output pairs. This is highly reminiscent of machine learning: given training data provided as input-output pairs, we find a program that matches inputs to outputs and can generalize to new inputs. The difference is that instead of learning parameter values in a hardcoded program (a neural network), we generate source code via a discrete search process (see table 19.2).

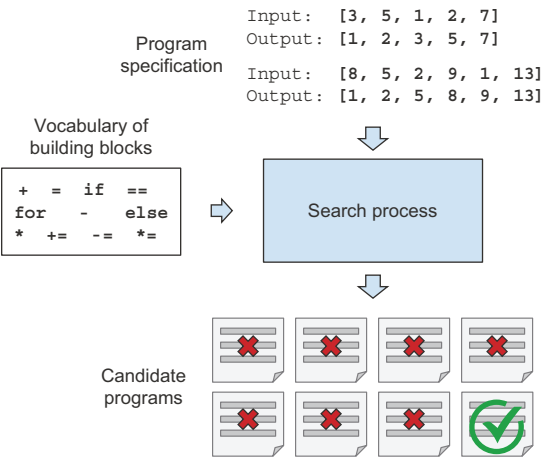


Figure 19.14 A schematic view of program synthesis: given a program specification and a set of building blocks, a search process assembles the building blocks into candidate programs, which are then tested against the specification. The search continues until a valid program is found.

Table 19.2 Machine learning vs. program synthesis

Machine learning	Program synthesis
Model: differentiable parametric function	Model: graph of operators from a programming language
Engine: gradient descent	Engine: discrete search (such as genetic search)
Requires a lot of data to produce reliable results	Data efficient: can work with a couple of training examples

Program synthesis is how we’re going to add program-centric abstraction capabilities to our AI systems. It’s the missing piece of the puzzle.

19.4.5 *Blending deep learning and program synthesis*

Of course, deep learning isn't going anywhere. Program synthesis isn't its replacement; it is its complement. It's the hemisphere that has been so far missing from our artificial brains. We're going to be using both, in combination. There are two major ways this will take place:

- Developing systems that integrate both deep learning modules and discrete algorithmic modules
- Using deep learning to make the program search process itself more efficient

Let's review each of these possible avenues.

INTEGRATING DEEP LEARNING MODULES AND ALGORITHMIC MODULES INTO HYBRID SYSTEMS

Today, many of the most powerful AI systems are hybrid: they use both deep learning models and handcrafted symbol-manipulation programs. In DeepMind's AlphaGo, for example, most of the intelligence on display is designed and hardcoded by human programmers (such as Monte Carlo Tree Search). Learning from data happens only in specialized submodules (value networks and policy networks). Or consider the Waymo self-driving car: it's able to handle a large variety of situations because it maintains a model of the world around it—a literal 3D model—full of assumptions hardcoded by human engineers. This model is constantly updated via deep learning perception modules (powered by Keras) that interface it with the surroundings of the car.

For both of these systems—AlphaGo and self-driving vehicles—the combination of human-created discrete programs and learned continuous models is what unlocks a level of performance that would be impossible with either approach in isolation, such as an end-to-end deep net or a piece of software without machine learning elements. So far, the discrete algorithmic elements of such hybrid systems are painstakingly hardcoded by human engineers. But in the future, such systems may be fully learned, with no human involvement.

What will this look like? Consider a well-known type of network: recurrent neural networks (RNNs). It's important to note that RNNs have slightly fewer limitations than feedforward networks. That's because RNNs are a bit more than mere geometric transformations: they're geometric transformations *repeatedly applied inside a `for` loop*. The temporal `for` loop is itself hardcoded by human developers: it's a built-in assumption of the network. Naturally, RNNs are still extremely limited in what they can represent, primarily because each step they perform is a differentiable geometric transformation, and they carry information from step to step via points in a continuous geometric space (state vectors). Now imagine a neural network that's augmented in a similar way with programming primitives but instead of a single hardcoded `for` loop with hardcoded continuous-space memory, the network includes a large set of programming primitives that the model is free to manipulate to expand its processing function, such as `if` branches, `while` statements, variable creation, disk storage for long-term memory, sorting operators, advanced data structures (such as lists, graphs, and hash tables), and many more. The space of programs that such a network could represent would be far

broader than what can be represented with current deep learning models, and some of these programs could achieve superior generalization power. Importantly, such programs will not be differentiable end to end, although specific modules will remain differentiable, and thus will need to be generated via a combination of discrete program search and gradient descent.

We'll move away from having, on one hand, hardcoded algorithmic intelligence (handcrafted software) and, on the other hand, learned geometric intelligence (deep learning). Instead, we'll have a blend of formal algorithmic modules that provide reasoning and abstraction capabilities and geometric modules that provide informal intuition and pattern-recognition capabilities (figure 19.15). The entire system will be learned with little or no human involvement. This should dramatically expand the scope of problems that can be solved with machine learning—the space of programs that we can generate automatically, given appropriate training data. Systems like AlphaGo—or even RNNs—can be seen as a prehistoric ancestor of such hybrid algorithmic-geometric models.

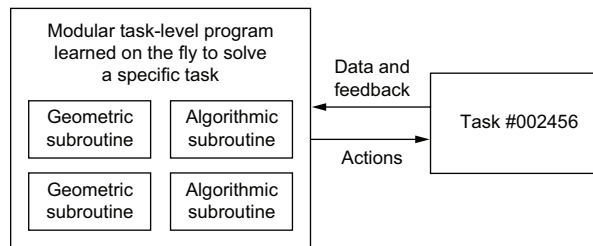


Figure 19.15 A learned program relying on both geometric primitives (pattern recognition, intuition) and algorithmic primitives (reasoning, search, memory)

USING DEEP LEARNING TO GUIDE PROGRAM SEARCH

Today, program synthesis faces a major obstacle: it's tremendously inefficient. To caricature, typical program synthesis techniques work by trying every possible program in a search space until it finds one that matches the specification provided. As the complexity of a program specification increases, or as the vocabulary of primitives used to write programs expands, the program search process runs into what's known as *combinatorial explosion*: the set of possible programs to consider grows very fast, in fact, much faster than merely exponentially fast. As a result, today, program synthesis can only be used to generate very short programs. You're not going to be generating a new OS for your computer anytime soon.

To move forward, we're going to need to make program synthesis efficient by bringing it closer to the way humans write software. When you open your editor to code up a script, you're not thinking about every possible program you could potentially write. You only have in mind a handful of possible approaches: you can use your understanding of the problem and your past experience to drastically cut through the space of possible options to consider.

Deep learning can help program synthesis do the same: although each specific program we'd like to generate might be a fundamentally discrete object that performs non-interpolative data manipulation, evidence so far indicates that *the space of all useful programs* may look a lot like a continuous manifold. That means that a deep learning model that has been trained on millions of successful program-generation episodes might start to develop solid *intuition* about the *path through program space* that the search process should take to go from a specification to the corresponding program—just like a software engineer might have immediate intuition about the overall architecture of the script they're about to write and about the intermediate functions and classes they should use as stepping stones on the way to the goal.

Remember that human reasoning is heavily guided by value-centric abstraction—that is, by pattern recognition and intuition. The same should be true of program synthesis. I expect the general approach of guiding program search via learned heuristics to see increasing research interest over the next 10 to 20 years.

19.4.6 *Modular component recombination and lifelong learning*

If models become more complex and are built on top of richer algorithmic primitives, then this increased complexity will require higher reuse between tasks, rather than training a new model from scratch every time we have a new task or a new dataset. Many datasets don't contain enough information for us to develop a new, complex model from scratch, and it will be necessary to use information from previously encountered datasets (much as you don't learn English from scratch every time you open a new book—that would be impossible). Training models from scratch on every new task is also inefficient due to the large overlap between the current tasks and previously encountered tasks.

With modern foundation models, we're starting to move closer to a world where AI systems possess enormous amounts of acquired knowledge and skills and can bring them to bear on whatever comes their way. But LLMs are missing a key ingredient: recombination. LLMs are very good at fetching and reapplying memorized functions, but they're not yet able to recombine those functions on the fly into brand-new programs adapted to the situation at hand. They are, in fact, entirely incapable of performing function composition, as investigated in a recent paper by Dziri et al.⁵ What's more, the kind of functions they learn aren't sufficiently abstract or modular, making them a poor fit for recombination in the first place. Remember how we pointed out that LLMs have low accuracy when adding large integers? You probably wouldn't want to build your next codebase on top of such brittle functions.

To solve *compositional generalization*, we're going to need to reuse robust *program components* like the functions and classes found in human programming languages. These components will be evolved specifically for modular reuse in a new context—unlike the patterns that LLMs memorize. And our AIs will recombine them on the fly to synthesize

⁵ Nouha Dziri et al., “Faith and Fate: Limits of Transformers on Compositionality,” Proceedings of the 37th International Conference on Neural Information Processing Systems (2023), <https://arxiv.org/abs/2305.18654>.

new programs adapted to the current task. Crucially, libraries of such reusable components will be built through the cumulative experience of all instances of our AIs and will then be accessible by all in perpetuity. Any single problem encountered by our AIs would only need to be solved once—making them constantly self-improving.

Think of the process of software development today: once an engineer solves a specific problem (HTTP queries in Python, for instance), they package it as an abstract, reusable library, accessible by anyone on the planet. Engineers who face a similar problem in the future will be able to search for existing libraries, download one, and use it in their own project. In a similar way, in the future, meta-learning systems will be able to assemble new programs by sifting through a global library of high-level reusable blocks. When the system finds itself developing similar program subroutines for several different tasks, it can come up with an *abstract*, reusable version of the subroutine and store it in the global library (see figure 19.16). These subroutines can be either geometric (deep learning modules with pretrained representations) or algorithmic (closer to the libraries that contemporary software engineers manipulate).

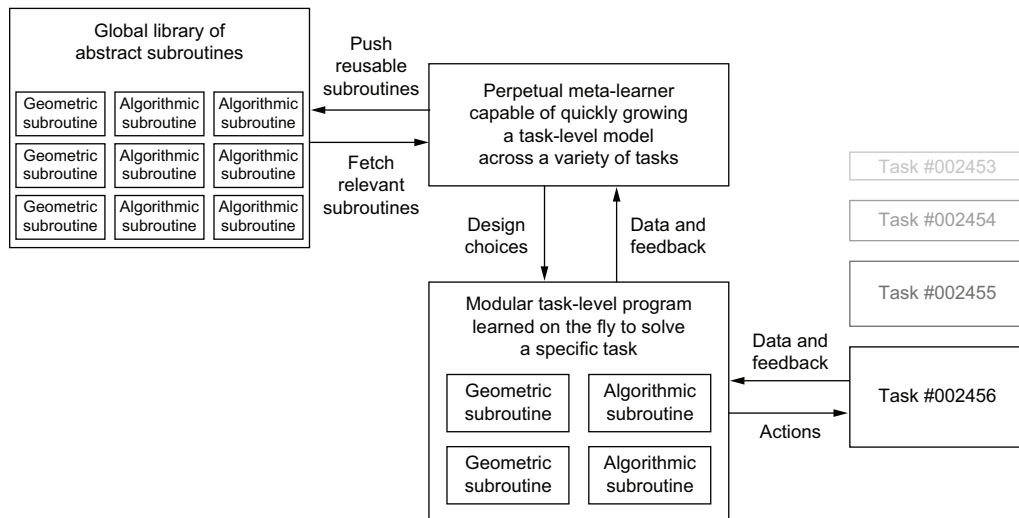


Figure 19.16 A meta-learner capable of quickly developing task-specific models using reusable primitives (both algorithmic and geometric), thus achieving extreme generalization

19.4.7 The long-term vision

In short, here's my long-term vision for AI:

- Models will be more like programs and will have capabilities that go far beyond the continuous geometric transformations of the input data we currently work with. These programs will arguably be much closer to the abstract mental models

that humans maintain about their surroundings and themselves, and they will be capable of stronger generalization due to their rich algorithmic nature.

- In particular, models will blend *algorithmic modules* providing formal reasoning, search, and abstraction capabilities with *geometric modules* providing informal intuition and pattern-recognition capabilities. This will achieve a blend of value-centric and program-centric abstraction. AlphaGo or self-driving cars (systems that required a lot of manual software engineering and human-made design decisions) provide an early example of what such a blend of symbolic and geometric AI could look like.
- Such models will be *grown* automatically rather than hardcoded by human engineers, using modular parts stored in a global library of reusable subroutines—a library evolved by learning high-performing models on thousands of previous tasks and datasets. As frequent problem-solving patterns are identified by the meta-learning system, they will be turned into reusable subroutines—much like functions and classes in software engineering—and added to the global library.
- The process that searches over possible combinations of subroutines to grow new models will be a discrete search process (program synthesis), but it will be heavily guided by a form of *program-space intuition* provided by deep learning.
- This global subroutine library and associated model-growing system will be able to achieve some form of human-like *extreme generalization*: given a new task or situation, the system will be able to assemble a new working model appropriate for the task using very little data, thanks to rich program-like primitives that generalize well, and extensive experience with similar tasks. In the same way, humans can quickly learn to play a complex new video game if they have experience with many previous games because the models derived from this previous experience are abstract and program-like, rather than a basic mapping between stimuli and action.

This perpetually learning model-growing system can be interpreted as an *artificial general intelligence* (AGI). But don't expect any singularitarian robot apocalypse to ensue: that's pure fantasy, coming from a long series of profound misunderstandings of both intelligence and technology. Such a critique, however, doesn't belong in this book.